

Here is the complete engineering blueprint for your ultra-low-power, false-alarm-proof balcony security node.

1. Hardware Selection & Engineering Analysis

LDO Regulator: MCP1700-3302E vs. HT7333-A

You should use the MCP1700-3302E.

- The MCP1700-3302E has an ultra-low typical quiescent current of **1.6 μ A**.
- The HT7333-A has a higher typical quiescent current of roughly **3 μ A**.
- *Crucial Engineering Fix:* The MCP1700-3302E is rated for a maximum of 250mA continuous output, but the ESP32-C3 can spike to ~350mA during short WiFi transmissions. To prevent the ESP32-C3 from browning out and rebooting, you **must** place a large electrolytic capacitor (470 μ F to 1000 μ F) across the 3.3V and GND pins of the ESP32-C3. This acts as a localized battery to absorb the WiFi TX spikes.

PIR Sensor: AM312 for Balcony Use?

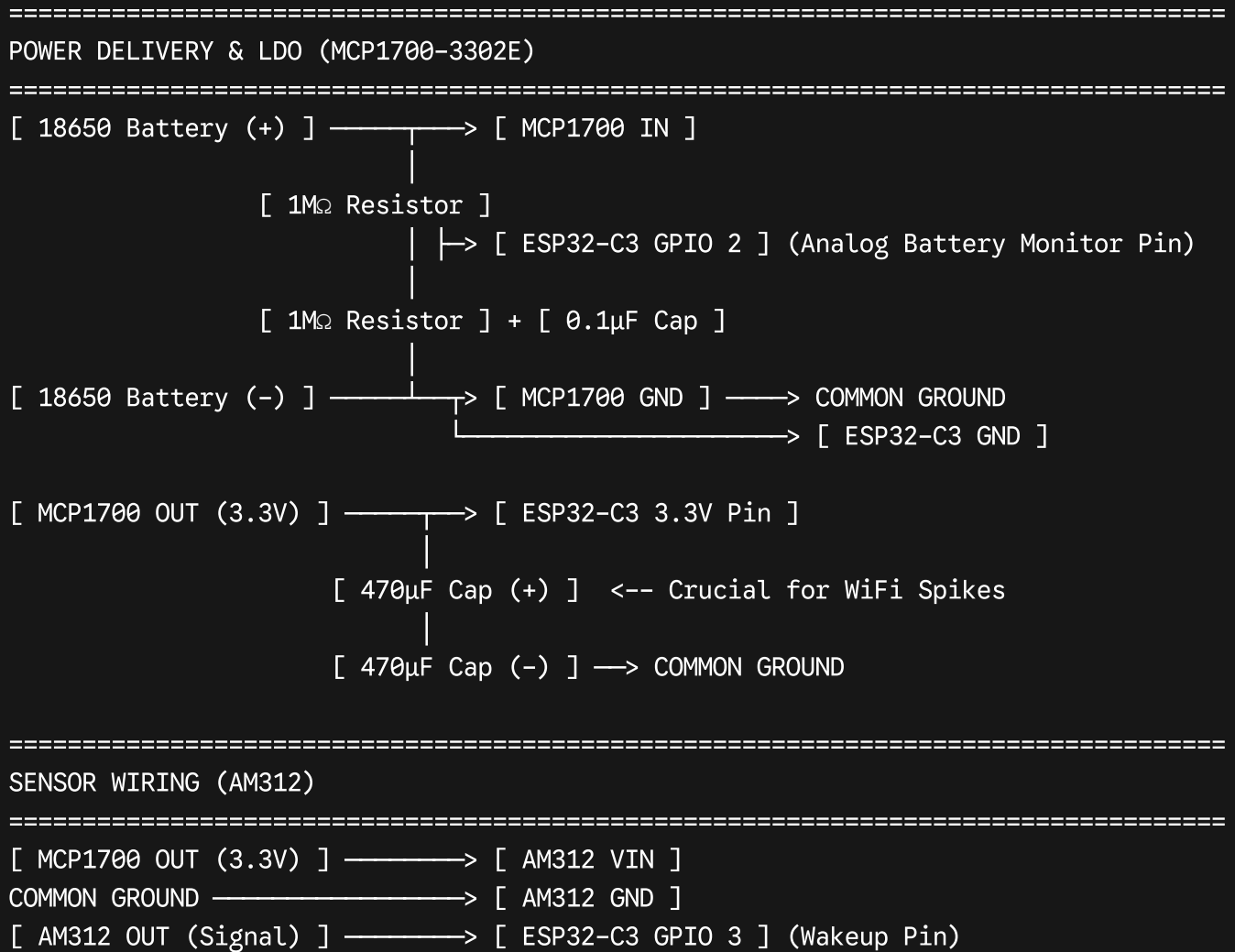
Yes, the AM312 is excellent for battery life (consuming only ~15 μ A), but **out of the box, it will cause severe false alarms on a balcony**. PIR sensors detect sudden changes in infrared heat. Direct sunlight, hot wind gusts, or reflections from glass will trigger it.

- *The False-Alarm-Proof Solution (Curtain Masking):* You must convert it into a "Curtain Sensor." Take a small piece of dark PVC pipe (about 1 inch long) or use black electrical tape to cover the entire white dome *except* for a narrow vertical slit. Mount the sensor so this slit looks strictly parallel to your balcony railing, angled slightly downward. This prevents it from seeing into your room, seeing the sky, or catching moving shadows.

2. The Ultra-Low Power Bill of Materials (BOM)

- **Microcontroller:** ESP32-C3 SuperMini or standard C3 Dev Board (Note: You must remove the onboard power LED and the factory LDO if you are using a dev board, as they drain power).
- **Power Source:** 1x Used 18650 Lithium-Ion Battery (No TP4056 charging circuit included in this build to save space/complexity. You will manually swap/charge it every 1-2 years).
- **Voltage Regulator:** MCP1700-3302E (Steps the 3.7V-4.2V battery down to a safe 3.3V for the ESP32).
- **Capacitor:** 1x 470 μ F or 1000 μ F Electrolytic Capacitor (Mandatory for WiFi stability).
- **Sensor:** AM312 Mini PIR Motion Sensor.
- **Telemetry:** 2x 1M Ω Resistors + 1x 0.1 μ F Ceramic Capacitor (For battery voltage monitoring without draining the battery).

3. Circuit Wiring Diagram



Expected Power Draw:

ESP32-C3 (Deep Sleep: $\sim 5\mu\text{A}$) + MCP1700 ($\sim 1.6\mu\text{A}$) + AM312 ($\sim 15\mu\text{A}$) + Resistor Divider ($\sim 2\mu\text{A}$)
 = **$\sim 23.6\mu\text{A}$ total standby.**

A used 1500mAh 18650 battery will easily last over 1.5 years on a single charge.

4. ESP32-C3 Firmware (Fast-Connect & Deep Sleep)

To maximize battery life, this code skips standard DHCP IP assignment (which takes 2-4 seconds) and uses a **Static IP**, allowing the ESP32 to wake up, connect, fire the MQTT message, and go back to sleep in under **300 milliseconds**.

```

#include <WiFi.h>
#include <PubSubClient.h>
#include <esp_sleep.h>

```

```

// --- NETWORK SETTINGS (STATIC IP IS MANDATORY FOR BATTERY LIFE) ---
const char* ssid = "YOUR_WIFI_SSID";
const char* password = "YOUR_WIFI_PASSWORD";
const char* mqtt_server = "192.168.1.100"; // Your Raspberry Pi IP

IPAddress local_IP(192, 168, 1, 201); // Assign a static IP for this sensor
IPAddress gateway(192, 168, 1, 1);
IPAddress subnet(255, 255, 255, 0);

WiFiClient espClient;
PubSubClient client(espClient);

// --- PIN DEFINITIONS ---
#define WAKEUP_PIN GPIO_NUM_3
#define BATTERY_PIN 2

void setup() {
    // 1. Immediately check what woke us up
    esp_sleep_wakeup_cause_t wakeup_reason = esp_sleep_get_wakeup_cause();

    if (wakeup_reason == ESP_SLEEP_WAKEUP_GPIO) {
        // 2. Read Battery Voltage
        int adc_value = analogRead(BATTERY_PIN);
        float voltage = (adc_value * 3.3 / 4095.0) * 2.0; // 2.0 is the resistor divider
        int battery_pct = map(voltage * 100, 330, 420, 0, 100);
        battery_pct = constrain(battery_pct, 0, 100);

        // 3. Connect to WiFi via Static IP (Ultra-Fast)
        WiFi.config(local_IP, gateway, subnet);
        WiFi.begin(ssid, password);

        int retries = 0;
        while (WiFi.status() != WL_CONNECTED && retries < 20) {
            delay(50);
            retries++;
        }

        // 4. Send MQTT Payload if Connected
        if (WiFi.status() == WL_CONNECTED) {
            client.setServer(mqtt_server, 1883);
            if (client.connect("BalconySensor")) {

                // Publish Motion Event
                client.publish("home/balcony/motion", "ON");

                // Publish Battery Level
                char batStr[8];
                snprintf(batStr, sizeof(batStr), "%d", battery_pct);
                client.publish("home/balcony/battery", batStr);
            }
        }
    }
}

```

```

        delay(10); // Brief pause to ensure packet delivery
    }
}

// 5. Configure GPIO Wakeup and Sleep
// ESP32-C3 uses this specific function to wake up on HIGH
esp_deep_sleep_enable_gpio_wakeup(1ULL << WAKEUP_PIN, ESP_GPIO_WAKEUP_GPIO_HIGH);

// Disable WiFi to save power before sleep
WiFi.mode(WIFI_OFF);

// Go into Deep Sleep
esp_deep_sleep_start();
}

void loop() {
    // Empty. The ESP32-C3 never reaches here. It only sleeps and resets.
}

```

5. Home Assistant Logic (The "Brain")

This system relies on Home Assistant (hosted on your RPi) to process the logic. The sensor *always* sends a motion trigger, but Home Assistant decides whether to trigger the siren and call the police based on whether you are home or away.

Step A: Define MQTT Sensors in `configuration.yaml`

YAML



```

mqtt:
  binary_sensor:
    - name: "Balcony Motion"
      state_topic: "home/balcony/motion"
      off_delay: 30 # Automatically resets to "Clear" after 30 seconds in the dashbo
      device_class: motion

  sensor:
    - name: "Balcony Sensor Battery"
      state_topic: "home/balcony/battery"
      unit_of_measurement: "%"
      device_class: battery

```

Step B: The Automation Logic (Armed vs. Disarmed)

Go to Home Assistant Automations and create a new automation.

- **Trigger:** When `binary_sensor.balcony_motion` turns `ON`.
- **Condition:** Ensure the alarm is Armed. (e.g., `alarm_control_panel.home_alarm` is `armed_away` OR a custom `input_boolean.balcony_guard` is `ON`). If this condition is not met (i.e., you are home in the room), the automation stops here. No sirens, no calls.
- **Actions (If Armed):**
 1. **Trigger Master Node Siren:** Send an MQTT payload to `home/master_node/siren/set` with the payload `ON`. (This immediately starts the Master Node).
 2. **Trigger RainMaker / Push Notification:** Send a critical push notification to your phone ("CRITICAL: Balcony Intrusion Detected!").
 3. **Trigger RPi SIM Module:** Call a service (e.g., a python script or shell command linked to your SIM800L module) to begin iterating through the SOS phone numbers and firing off the emergency SMS texts.